

# 软件体系结构

## 1 软件体系结构的发展

(略)

## 2 理解软件体系结构

### 2.1 概念和定义

#### 软件体系结构的定义

软件体系结构目前还没有统一的定义，常见的定义是[Shaw1995]提出的：软件体系结构 = { 部件(Component), 连接件(Connector), 配置(Configuration)}。其中：

1. “部件”是软件体系结构的基本组成单位之一，承载系统的主要功能，包括处理与数据；
2. “连接件”是软件体系结构的另一个基本组成单位，定义了部件间的交互，是连接的抽象表示
3. “配置”是对“形式”的发展，定义了“部件”以及“连接件”之间的关联方式，将它们组织称系统的总体结构

简单来说，一个软件系统的体系结构规定了系统的计算部件和部件之间的交互。

需要注意的是：

1. 连接件是一个与部件平等的单位。在软件的详细设计中，交互与计算是交织在一起的，但是在软件体系结构中呢，连接件将交互从计算中独立出来进行抽象和封装。
2. 部件与连接件是比类、模块等软件单位更高层的抽象。部件和连接件也既有抽象规格，又拥有“模块+连接”的具体实现

### 2.2 区分物理和逻辑

物理和逻辑的区分也可以看作高层和低层的区分、抽象和实现的区分。逻辑是抽象层面的，面对用户的，与真正的物理实现可能有所不同，但对外表现的是用户期待的或者需求所需要的。

#### 物理实现的载体

- 低层：基本类型 + 基本控制结构
- 中层：OO编程语言机制
  - 类生命、实例创建与撤销、实例生命期管理
  - 类权限控制机制
  - 复杂机制：继承
- 高层：导入导出和名称匹配

什么是导入导出：

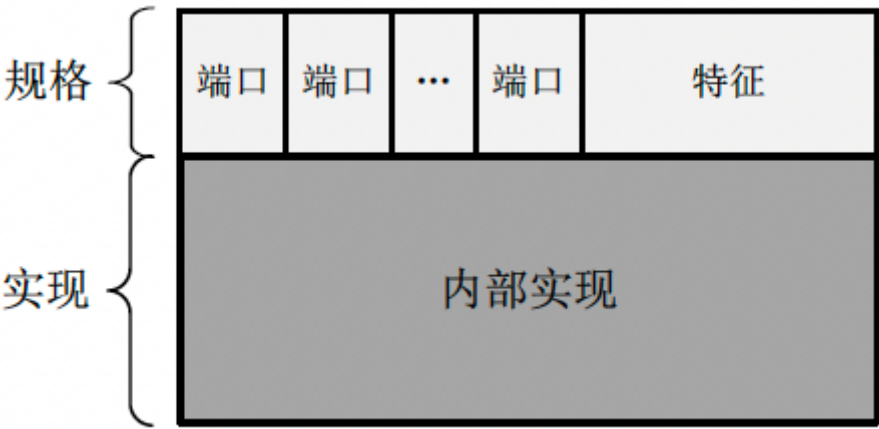
- 导入：比如java中的import
- 导出：比如使用包作为一个整体结构供其他类使用，比如声明为public供其他方法调用都是一种导出

## 2.3 高层抽象

### 2.3.1 部件

部件是一种包含了系统中的数据和处理的软件体系结构的一部分，通常只暴露接口而隐藏内部实现，可以分为原始（Primitive）部件和复合（Composite）部件两种类型

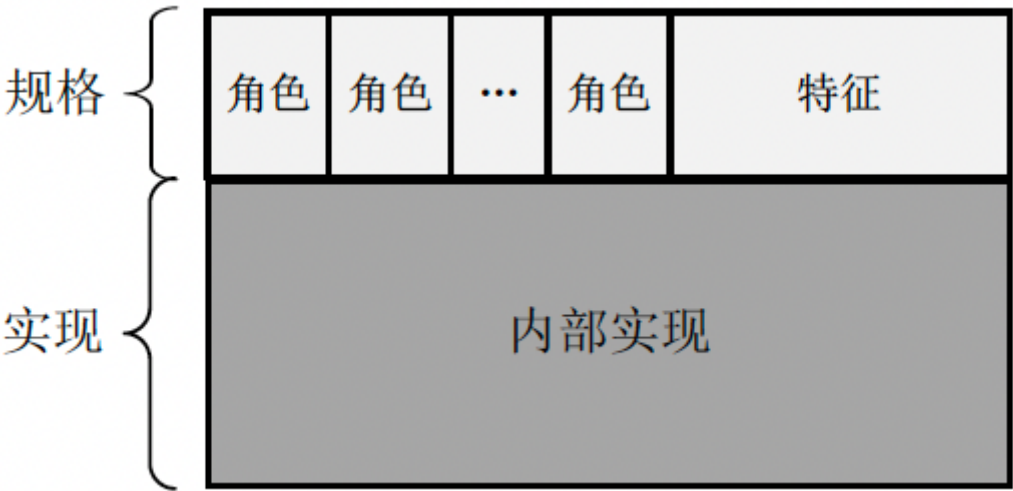
- 原始类型的部件可以直接被实现为相应的软件实现机制
- 复合部件则由更细粒度的部件和连接件组成，复合部件通过局部配置将其内部的部件和连接件连接起来，构成一个整体



### 2.3.2 连接件

连接件也可以分为原始（Primitive）连接件和复合（Composite）连接件两种类型。

1. 原始类型的连接件可以直接被实现为相应的软件实现机制。
2. 复合部件则由更细粒度的部件和连接件组成，复合部件通过局部配置将其内部的部件和连接件连接起来，构成一个整体

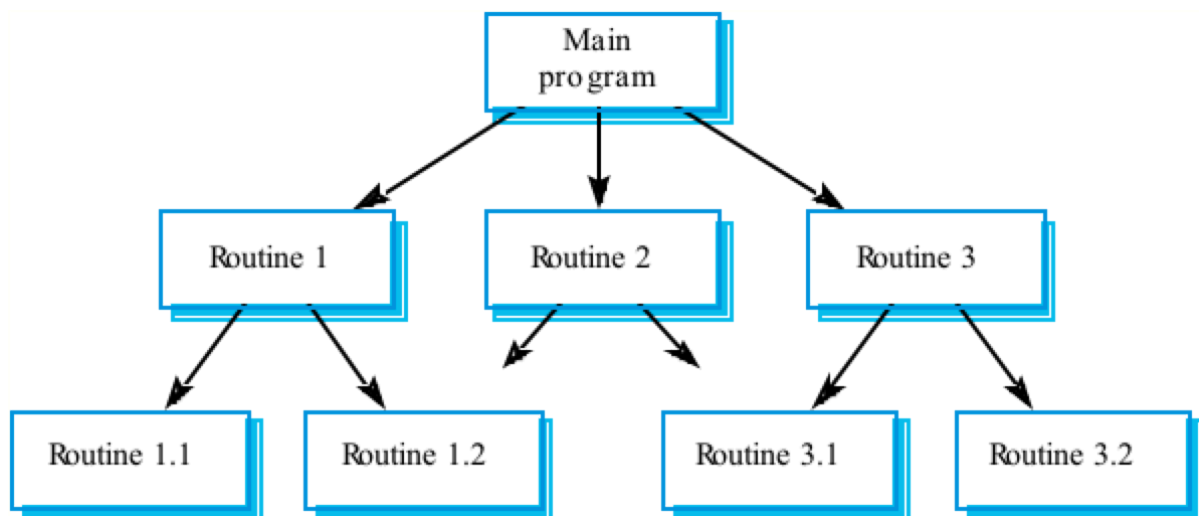


### 2.3.3 配置

配置是一种专门将部件和连接件连接起来的机制，构成系统的整体结构，达到系统的设计目标

## 3 体系结构风格初步

### 3.1 主程序/子程序



#### 设计决策与约束

1. 基于声明-使用(程序调用)关系建立连接件,以层次分解的方式建立系统部件,共同组成层次结构。
2. 每一个上层部件可以“使用”下层部件,但下层部件不能“使用”上层部件,即不允许逆方向调用。
3. 系统应该是单线程执行。主程序部件拥有最初的执行控制权,并在“使用”中将控制权转移给下层子程序。
4. 子程序只能通过上层转移来获得控制权,可以在执行中将控制权转交给下层的子程序,并在自身执行完成之后必须将控制权还交给上层部件。

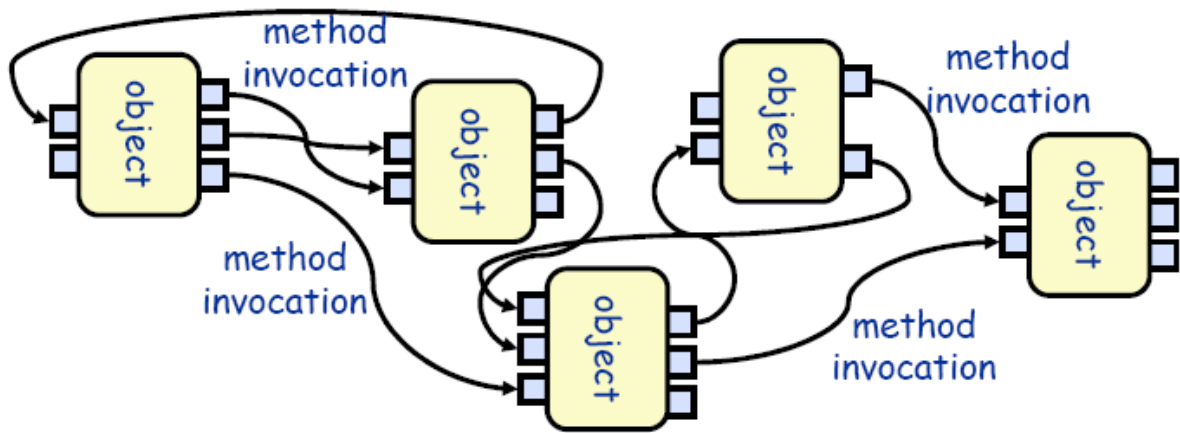
#### 3.1.1 优点

1. 流程清晰，易于理解：严格的层次分解使得整个系统的组织结构非常符合功能分解和分而治之的思维方式，从而能够清晰的描述整个系统的执行流程，易于理解
2. 强控制性：严格的层次分解和严格的控制权转移使得主程序/子程序风格对程序的实际执行过程具备很强的控制能力。

#### 3.1.2 缺点

1. 程序调用是一种强耦合的连接方式，非常依赖交互方的接口规格，这会使得系统难以修改和复用
2. 程序调用的连接方式限制了各部件之间的数据交互，可能会使得不同部件使用隐含的共享数据交流，产生不必要的公共耦合，进而破坏它的“正确性”控制能力。

## 3.2 面向对象式



### 设计决策与约束

1. 依照对数据的使用情况,用信息内聚的标准,为系统建立对象部件。每个对象部件基于内部数据提供对外服务接口,并隐藏内部数据的表示。
2. 基于方法调用(Method Invocation)机制建立连接件,将对象部件连接起来。
3. 每个对象负责维护其自身数据的一致性与完整性,并以此为基础对外提供“正确”的服务。
4. 每个对象都是一个自治单位,不同对象之间是平级的,没有主次、从属、层次、分解等关系。

### 3.2.1 优点

1. 内部实现的可修改性。因为面向对象式风格要求封装内部数据,隐藏内部实现,所以它可以在不影响外界的情况下,变更其内部实现。
2. 易开发、易理解、易复用的结构组织。面向对象式风格将系统组织为一系列平等、自治的单位,每个单位负责自身的正确性,不同单位之间仅仅是通过方法调用相连接,这非常契合模块化的思想,能够建立一个易开发、易理解、易服用的实现结构

### 3.2.2 缺点

1. 接口的耦合性。虽然面向对象式风格有利于对象修改自己的内部实现,但是其所用的方法调用连接机制使得它无法消除接口的耦合性
2. 标识 (Identity) 的耦合性。除了接口的耦合性之外,方法调用机制带来的还有标识的耦合性,即一个对象要与其他对象交互,就必须知道其他对象的标识
3. 副作用。例如A和B都使用C,那么B对A的修改可能会对A产生未预期的影响。

## 3.3 分层

### 设计决策与约束

1. 从最底层到最高层,部件的抽象层次逐渐提升。每个下层为邻接上层提供服务,每个上层将邻接下层作为基础设施使用。也就是说,在程序调用机制中上层调用下层。
2. 两个层次之间的连接要遵守特定的交互协议,该交互协议应该是成熟、稳定和标准化的。也就是说,只要遵守交互协议,不同部件实例之间是可以互相替换的。
3. 跨层次的连接是禁止的,不允许第 I 层直接调用  $I+N(N>1)$  层的服务。
4. 逆向的连接是禁止的,不允许第 I 层调用第  $J(J<I)$  层的服务。

### 3.3.1 优点

1. 设计机制清晰，易于理解。
2. 支持并行开发。
3. 更好的可复用性与内部可修改性

### 3.3.2 缺点

1. 交互协议难以修改
2. 性能损失。逐层调用可能会产生冗余的请求调用导致性能算户
3. 难以确定层次数量和粒度

## 3.4 MVC

### 设计决策和约束

1. 模型、视图、控制是分别是关于业务逻辑、表现和控制的三种不同内容抽象。
2. 如果视图需要持续地显示某个数据的状态,那么它首先需要在模型中注册对该数据的兴趣。如果该数据状态发生了变更,模型会主动通知视图,然后再由视图查询数据的更新情况。
3. 视图只能使用模型的数据查询服务,只有控制部件可以调用可能修改模型状态的程序。
4. 用户行为虽然由视图发起,但是必须转交给控制部件处理。对接收到的用户行为,控制部件可能会执行两种处理中的一种或两种:调用模型的服务,执行业务逻辑;提供下一个业务展现。模型部件相对独立,既不依赖于视图,也不依赖于控制。虽然模型与视图之间存在一个“通知变更”的连接,但该连接的交互协议是非常稳定的,可以认为是非常弱的依赖。

### 3.4.1 优点

1. 易开发性。试图和控制的可修改性
2. 模型封装了系统的业务逻辑，所以是三种类型中最为复杂的系统部件。MVC中模型是相对独立的，所以对视图实现和控制实现的修改不会影响到模型实现。再考虑到业务逻辑通常比业务表现和控制逻辑更加稳定，所以MVC具有一定的可修改性优势
3. 适用于网络系统开发的特征。MVC不仅允许视图和控制的可修改性,而且其对业务逻辑、表现和控制的分离使得一个模型可以同时建立并保持多个视图,这非常适用于网络系统开发。

### 3.4.2 缺点

1. 复杂性。MVC 将用户任务分解成了表现、控制和模型三个部分，这增加了系统的复杂性，不利于理解任务的实现
2. 模型修改困难。视图和控制都要依赖于模型，因此模型难以修改